

I-Présentation du langage et quelques règles d'écriture

I-1 Un exemple de programme en langage C

I-2 Structure d'un programme en langage C

I-3 Les identificateurs

I-4 Les mots clés

I-5 Les commentaires

I-6 Déclarations des variables

I-6-1 Les caractères

I-6-2 Les entiers

I-6-3 Les réels

I-6-4 Les constantes

I-7 La fonction Printf

I-8 La fonction Scanf

I-1 Un exemple de programme en langage C

Un programme en langage C, comporte des plusieurs blocs (des instructions, des fonctions) et des structures de haut niveau. Chaque bloc à un rôle dans la compilation, l'édition ou l'exécution.

Exemple:

```
# include <stdio.h>
# define TVA 18.6
Void main (void)
{
    Float HT, TTC;
    Printf ("veuillez entrer le prix HT");
    Scanf ("%F", &HT);
    TTC= HT*(1+(TVA/100));
    Printf ("prix TTC %Fxn",TTC);
}
```

on trouve dans le programme:

Les directives de précompilation

Elles commencent toutes par un #.

commande	signification
<code>#include <stdio.h></code>	<i>permet d'utiliser les fonctions printf() et scanf()</i>
<code>#include <math.h></code>	<i>permet d'utiliser les fonctions mathématiques</i>
<code>#define PI 3.14159</code>	<i>définit la constante PI</i>
<code>#undef PI</code>	<i>à partir de cet endroit, la constante PI n'est plus définie</i>
<code>#ifdef PI</code> <i>instructions 1 ...</i> <code>#else</code> <i>instructions 2 ...</i> <code>#endif</code>	<i>si la constante PI est définie, on compile les instructions 1, sinon, les instructions 2</i>

Parmi ces directives, une seule est obligatoire pour le bon fonctionnement d'un programme :

`#include <stdio.h>` . En effet, sans elle, on ne peut pas utiliser les fonctions utiles pour l'affichage à l'écran : `printf()` et la lecture des données au clavier : `scanf()`. Nous verrons le fonctionnement de ces fonctions plus tard.

La fonction main()

Elle commence par une accolade ouvrante { et se termine par une accolade fermante }. À l'intérieur, chaque instruction se termine par un point-virgule. Toute variable doit être déclarée.

I-2 Structure du langage C:

Un programme C se présente de la façon suivante:

```
[directive préprocesseur];  
[déclaration de variables];  
Main ()  
{  
    bloc d'instruction;  
}
```

I-3 Les identificateurs

Les identificateurs servent à désigner les différents « objets » manipulés par le programme :

variables, fonctions, etc. (Nous rencontrerons ultérieurement les autres objets manipulés par le langage C : constantes, étiquettes de structure, d'union ou d'énumération, membres de structure

ou d'union, types, étiquettes d'instruction GOTO, macros). Comme dans la plupart des langages, ils sont formés d'une suite de caractères choisis parmi les **lettres** ou les **chiffres**, le premier d'entre eux étant nécessairement une lettre.

En ce qui concerne les lettres :

le caractère souligné (`_`) est considéré comme une lettre. Il peut donc apparaître au début d'un identificateur. Voici quelques identificateurs corrects :

`lg_lig` `valeur_5` `_total` `_89`

I-4 Les mots-clés

Certains « mots-clés » sont réservés par le langage à un usage bien défini et ne peuvent pas être utilisés comme identificateurs. En voici la liste, classée par ordre alphabétique.

Les mots-clés du langage C

<code>auto</code>	<code>default</code>	<code>float</code>	<code>register</code>	<code>struct</code>	<code>volatile</code>
<code>break</code>	<code>do</code>	<code>for</code>	<code>return</code>	<code>switch</code>	<code>while</code>
<code>case</code>	<code>double</code>	<code>goto</code>	<code>short</code>	<code>typedef</code>	
<code>char</code>	<code>else</code>	<code>if</code>	<code>signed</code>	<code>union</code>	
<code>const</code>	<code>enum</code>	<code>int</code>	<code>sizeof</code>	<code>unsigned</code>	
<code>continue</code>	<code>extern</code>	<code>long</code>	<code>static</code>	<code>void</code>	

I-5 Les commentaires

Un commentaire débute par `/*` et se termine par `*/`. Par exemple,

```
/* Ceci est un commentaire */
```

On ne peut pas imbriquer des commentaires. Quand on met en commentaire un morceau de programme, il faut donc veiller à ce que celui-ci ne contienne pas de commentaire.

Voici quelques exemples de commentaires :

```
/* programme de calcul de racines carrées */

/* commentaire fantaisiste &ç${<>} ?%!!!!!! */

/* commentaire s'étendant
sur plusieurs lignes
de programme source */

/* =====
*      commentaire quelque peu esthétique      *
*      et encadré, pouvant servir,                *
*      par exemple, d'en-tête de programme      *
===== */
```

I-6 Déclaration des variables

La déclaration d'une variable consiste donc, dans un premier temps, à préciser au compilateur le type de la variable que l'on va utiliser, et donc le mode de codage qu'il va devoir effectuer pour le stockage. Ensuite, en fonction du type de la variable, il va réserver une place mémoire plus ou moins importante (selon qu'il s'agit d'un short ou d'un long).

Ces déclarations s'effectuent simplement en donnant le type de la variable suivi du nom de la variable en question :

```
int i;  
float x;  
double y;  
char c;
```

Lorsque l'on souhaite déclarer plusieurs variables du même type, on peut regrouper la déclaration sur une seule ligne:

```
int i,j,k;  
float x,y;
```

Initialisation des variables

La déclaration des variables se contente de signaler le type de la variable et de réserver, le cas échéant, de la mémoire. Dans ce dernier cas, la réservation de mémoire ne modifie pas son contenu précédent. Ainsi, la variable prend la valeur associée au contenu de la mémoire : ,ca peut être n'importe quoi. Il faut donc toujours initialiser une variable avant de l'utiliser. Ainsi,

```
int i;  
i = 0;
```

déclare une variable entière i, puis l'initialise à 0. La déclaration et l'initialisation peuvent être faites en une seule étapes au lieu de deux :

```
int i = 0;
```

Commençons donc par les types les plus simples et les plus utiles, à savoir les entiers, les flottants et les caractères.

1-6-1 Les variables entiers (int, short int, long int)

Les entiers sont codés à l'aide de la représentation des nombres entiers relatifs : un bit est réservé pour coder le signe, les autres bits codent la valeur absolue du nombre. Le langage C peut gérer plusieurs tailles différentes d'entiers :

- **int** (l'entier par défaut)
- **short int** ou simplement **short**
- **long int** ou simplement **long**

Type	Espace
short	[-32 768, 32 767]
int/long	[-2 147 483 648, 2 147 483 647]
long long	[-9 223 372 036 854 775 808, 9 223 372 036 854 775 807]
unsigned short	[0, 65 535]
unsigned int/long	[0, 4 294 967 295]
unsigned long long	[0, 18 446 744 073 709 551 615]

1-6-2 Les variables flottants (float, double et long double)

Le C propose trois tailles pour représenter ces flottants, avec une mantisse et un exposant de tailles variables :

- **float**
- **double**, le flottant en double précision
- **long double**, un type récent qui propose une précision quadruple.

Comme pour les entiers, les tailles et précisions des flottants nous avons :

- pour les float, une mantisse sur 24 bits et un exposant sur 8 bits, proposant une précision de l'ordre de 10^{-7} ;
- pour les double, une mantisse sur 53 bits et un exposant sur 11 bits, proposant une précision de l'ordre de 2×10^{-16} ;
- pour les long double, une mantisse sur 64 bits et un exposant sur 16 bits, proposant une précision de l'ordre de 10^{-19} .

Type	Mantisse	Précision	Min	Max
float	24	10^{-7}	10^{-38}	$3 \cdot 10^{38}$
double	53	$2 \cdot 10^{-16}$	$2 \cdot 10^{-308}$	10^{308}
long double	64	10^{-19}	10^{-4931}	10^{4932}

1-6-3 Variable caractère (char)

Les caractères sont essentiels, notamment pour écrire du texte, manipuler des lettres, etc. En C, ils sont représentés comme des entiers, sur un octet (entre 0 et 255) en utilisant la table de conversion ASCII. Cependant, cette table de conversion est transparente pour l'utilisateur, en notant le caractère entre apostrophes " ' ' ". Ainsi, la notation 'A' désignera le caractère " A ".

1-6-4 Constantes

Les constantes ont une syntaxe particulière, étant donné qu'il existe plusieurs types d'entiers et de flottants, puis qu'il existe également plusieurs notations.

Constantes de type entier

Les entiers sont construits à partir des chiffres (et des lettres pour une notation en hexadécimale). On ajoute alors éventuellement un suffixe " l " ou " L " pour désigner une constante de type long et/ou le suffixe " u " ou " U " pour désigner le qualificatif unsigned. Aucun préfixe signale que le nombre est exprimé en base 10, tandis que les préfixes 0 et 0x précisent respectivement une notation en octale ou en hexadécimale. Ainsi, 0377 désigne l'entier 255, 0xF désigne l'entier 15. En revanche, 120L désigne l'entier 120 de type long.

Constantes de type flottant

Les flottants peuvent bien sûr être désignés par des nombres à virgule (enfin, avec un point " . " !). On ajoute éventuellement un suffixe " f " ou " F " pour désigner une constante de type float ou le suffixe " l " ou " L " pour désigner le type long double. On peut également utiliser la notation scientifique 123E+3 ou -142E-4.

Constantes de type caractère

Les caractères sont simplement désignés entre apostrophes. Ainsi, la constante 'a' désigne le caractère a minuscule. On peut également noter ces lettres par leur code ASCII en utilisant les notations octales ou hexadécimales, précédées d'un backslash. Ceci permet de noter les caractères non affichables. Certains caractères admettent des abréviations :

'\101' et '\x41'	désignent le caractère A, de code ASCII 65, (également 'A')
'\0' et '\x0'	désignent le caractère nul (délimite la fin des chaînes de caractères)
'\12' et '\xA'	désignent le saut de ligne (également '\n')
'\13' et '\xB'	désignent le retour chariot (également '\r')

1-7 L'affichage avec printf

La fonction printf sert à afficher des données à l'écran. Les chaînes de caractères constantes s'affichent telles que, mais les autres constantes et les variables doivent être intégrées de façon particulière, spécifique au type et au format d'affichage.

Exemple:

```
#include <stdio.h>
int main ()
{
    int a,b,c;
    float x,y,z;
    printf("Entrer deux entiers : ");
    scanf("%d %d",&a,&b);
    printf("Entrer deux flottants : ");
    scanf("%f %f",&x,&y);
    c = a*b;
    z = x+y;
    printf("%5d * %5d = %5d \n",a,b,c);
    printf("%5.2f + %5.2f = %5.2f \n",x,y,z);
}
```

Ainsi, printf("Affichage"); écrit le mot Affichage à l'écran. Pour afficher le contenu d'une variable entière, on spécifie %d pour préciser qu'il s'agit d'un type entier, puis on précise le nom de la variable à insérer en fin :

```
printf("La variable i vaut %d",i);
```

Si on veut afficher la variable en octal ou en hexadécimal, on utilise %o et %x respectivement.

Pour les flottants, on utilise %f ou %e selon que l'on souhaite un affichage standard ou en notation exponentielle. Une liste plus complète peut être trouvée tableau ci-dessous montre affichage les entiers, les flottants et les chaînes de caractères.

Type et Variable	Affichage	Format
short i; int j; long k;	printf("%hd %d %ld",i,j,k);	décimal
short i; int j; long k;	printf("%ho %o %lo",i,j,k);	octal
short i; int j; long k;	printf("%hx %x %lx",i,j,k);	hexadécimal
float x; double y;	printf("%f %lf",x,y);	décimal (avec point)
float x; double y;	printf("%e %le",x,y);	exponentiel
float x; double y;	printf("%g %lg",x,y);	le plus court des deux
long double z;	printf("%Lf",z);	décimal (avec point)
long double z;	printf("%Le",z);	exponentiel
long double z;	printf("%Lg",z);	le plus court des deux
char c;	printf("%c",c);	caractère
char m[10];	printf("%s",m);	chaîne de caractères

Pour les flottants, il est de même possible de préciser le nombre de chiffres après la virgule, ainsi que le nombre total de caractères (point compris) :

```
printf("%6.2f %.3Lf",x,y);
```

affiche la variable flottante x avec 2 chiffres après la virgule, mais en insérant des espaces devant afin que le tout fasse 6 caractères. Tandis que le variable de type long double y sera simplement affichée avec 3 chiffres après la virgule, sans spécification de la longueur totale.

Enfin, il est également possible de formater l’affichage des chaînes de caractères en précisant la taille globale (le nombre d’espaces `a insérer en tête) :

```
printf("%20s",chaine);
```

affiche la chaîne de caractère chaine avec des espaces devant afin que le tout fasse 20 caractères.

1-8 La saisie avec scanf

La fonction scanf permet de saisir des valeurs tapées au clavier pendant l’exécution du programme. La syntaxe est tr`es semblable `a celle de la fonction printf, mais en sens inverse : scanf("valeurs %d et %d",&i,&j); attend, au clavier, une phrase de la forme valeurs 10 et 31 et mettra la valeur 10 dans la variable i et la valeur 31 dans la variable j. On remarquera que la notation des variables diffère de leur utilisation avec printf.

En effet, il faut indiquer `a la fonction scanf l’adresse de la variable dans laquelle on veut stocker la valeur scannée, d’ou le & qui accède au pointeur associe `a la variable qui suit.

Type et Variable	Affichage	Format
short i; int j; long k;	scanf("%hd",&i); scanf("%d",&j); scanf("%ld",&k);	décimal
short i; int j; long k;	scanf("%ho",&i); scanf("%o",&j); scanf("%lo",&k);	octal
short i; int j; long k;	scanf("%hx",&i); scanf("%x",&j); scanf("%lx",&k);	hexadécimal
float x; double y; long double z;	scanf("%f",&x); scanf("%lf",&y); scanf("%Lf",&z);	décimal (avec point)
float x; double y; long double z;	scanf("%e",&x); scanf("%le",&y); scanf("%Le",&z);	exponentiel
char c; char m[10];	scanf("%c",&c); scanf("%s",m); scanf("%s",&m[0]);	caractère chaîne de caractères